

Bluetooth Remote Control

1. Experimental Purpose

Based on the motor driver routine, use the YahboomRobot APP to control the car's movement status through the Bluetooth 5.0 module.

The states that can be controlled via the APP are as follows:

Stop	Forward	Backward
Turn Left	Turn Right	Spin Left
Spin Right		

2. Experimental Principle

Bluetooth Module

The Bluetooth module acts as a slave device, and the mobile phone APP acts as the master device for data transmission.

The Bluetooth module sends data from the mobile phone APP to the STM32F103C8T6 via TXD. The microcontroller then parses the received data to obtain the corresponding command data, thereby controlling the car's movement status.

The data format received by STM32F103C8T6 (refer to Yahboom mini program communication protocol):

Data Header + Data + Data Tail
\$ + Data + #

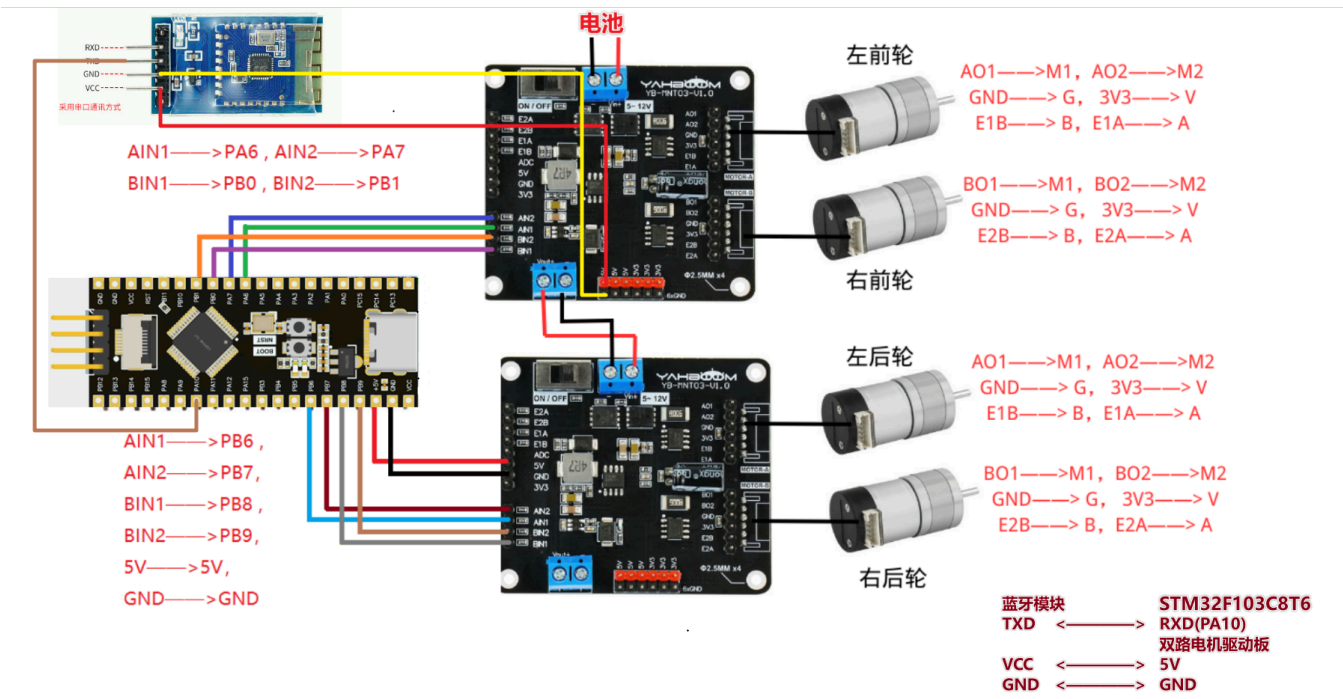
Motor Control

Two dual-channel motor driver boards are used to drive four motors. Refer to the motor driver routine for specific driving principles.

3. Hardware Connection

- Development Board:** STM32F103C8T6
- Modules:** Dual-channel motor driver board (AT8236), 310 motors (4), Bluetooth 5.0 module
- Connection:** Dupont wires (several)
- Power Supply:** Approximately 8V

Hardware Wiring Diagram



Bluetooth Module	STM32F103C8T6
TXD	RXD(PA10)
Dual-channel Motor Driver Board (AT8236)	
VCC	5V
GND	GND

Note:

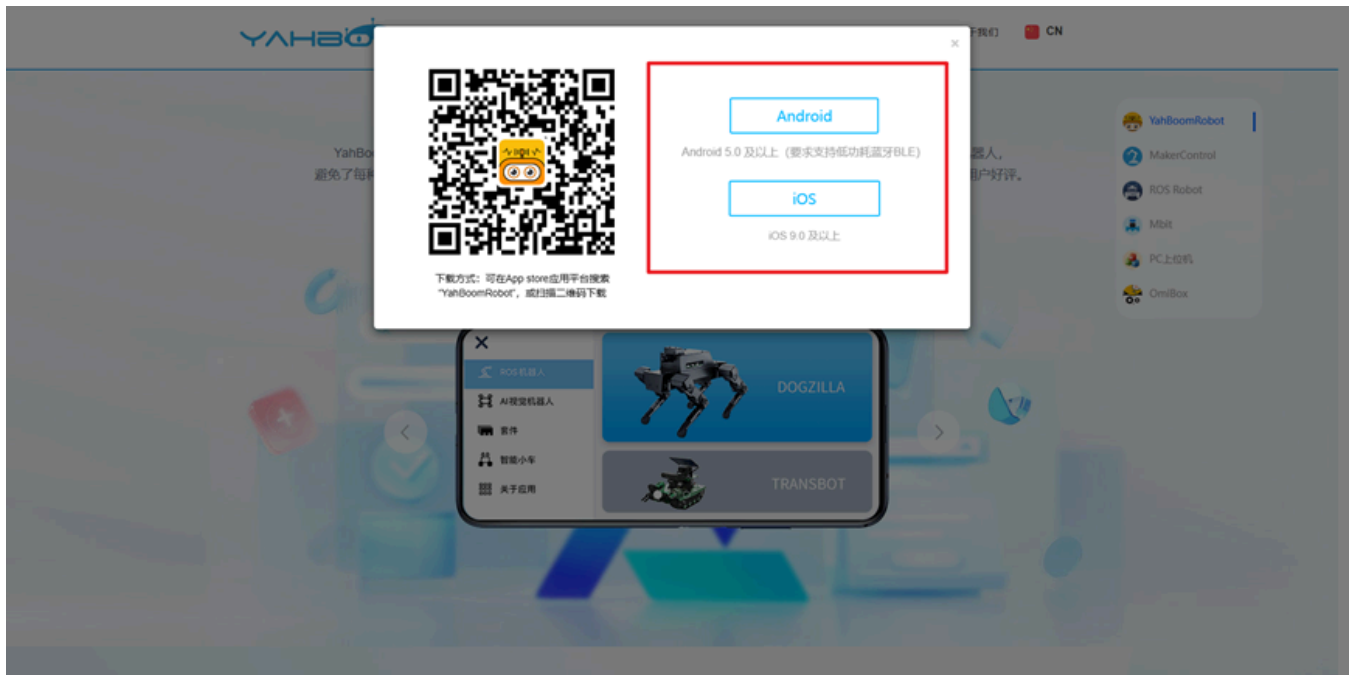
The 310 motor wiring must be connected according to the pin labels in the diagram. Do not connect arbitrarily or replace with pin headers yourself, otherwise the driver board will be burned!

After wiring is complete, upload the Bluetooth control code (during uploading, disconnect the PA10 wiring, then reconnect the Dupont wires after uploading is complete).

4. Bluetooth Connection

1. Download APP:

<https://www.yahboom.net/software/app>



2. YahboomRobot APP Specific Operations



5. Main Code

1. Code Files

Bluetooth control is implemented based on the motor driver. The motor driver routine files used are:

Motor.h, Motor.c, Pwm.h, Pwm.c

Two functions are added in the Motor.c file.

2. Implementation Principle

Use serial port interrupt to receive data and store correct data in an array.

First element in the array: Controls car stop, forward, backward, turn left, turn right

Second element in the array: Controls car spin left, spin right

3. Main Code

1.main.c

```
#include "stm32f10x.h"
#include "SysTick.h"
#include "Uart.h"
#include "Pwm.h"
#include "Motor.h"

int main(void)
{
    SysTick_Configuration();    //Configure timer
    Uart1_Configuration();      //Configure serial port
    Uart1_NVIC_Configuration(); //Configure serial port interrupt
    PWM_Int(7199,0, 7199,0);    //PWM initialization

    while(1)
    {
        Data_Analyse();          //Parse data received by serial port interrupt
        Car_Function(Car_state); //Control different car states
    }
}
```

2.Uart.c

```
void USART1_IRQHandler(void)
{
    uint8_t recv_dat = 0;
    static uint8_t rec_state = 0;
    while(USART_GetFlagStatus(USART1, USART_FLAG_RXNE) == SET)
    {
        recv_dat = USART_ReceiveData(USART1);

        switch(rec_state)
        {
            case 0:
                if((recv_dat == '$') && (!rx_index))
                {
                    rec_state = 1;
                    rx_index = 0;
                }
            else
            {
                rec_state = 0;
            }
        }
    }
}
```

```

        }
        break;
    case 1:
        if(recv_dat == '#')
        {
            rxd_flag = 1;
            rec_state = 0;
        }
        else
        {
            rxd_buf[rxd_index++] = recv_dat;
        }
        break;
    }
}
}

```

3.Motor.c

```

void Data_Analyse(void)          //Parse data received by serial port and stored in array
{
    if(rxd_flag == 1)
    {
        switch(rxd_buf[0])
        {
            case '1':
                printf("Forward!\n");
                Car_state = 1;
                break;
            case '2':
                printf("Backward!\n");
                Car_state = 2;
                break;
            case '3':
                printf("Left!\n");
                Car_state = 3;
                break;
            case '4':
                printf("Right!\n");
                Car_state = 4;
                break;
            case '0':
                printf("Stop!\n");
                Car_state = 0;
                break;
        }
        switch(rxd_buf[2])
        {
            case '1':
                printf("SpinLeft!\n");
                Car_state = 5;
                break;
            case '2':

```

```

        printf("SpinRight!\n");
        Car_state = 6;
        break;
    }
    rxd_flag = 0;
}

void Car_Function(unsigned int Car_state)    //Control car state
{
    switch(Car_state)
    {
        case 0:
            printf("Stop\n");
            Stop();
            break;
        case 1:
            printf("Forward\n");
            Forward(6000);
            break;
        case 2:
            printf("Backward\n");
            Backward(6000);
            break;
        case 3:
            printf("Turnleft\n");
            Turnleft(6500);
            break;
        case 4:
            printf("Turnright\n");
            Turnright(6500);
            break;
        case 5:
            printf("SpinLeft\n");
            SpinLeft(7000);
            break;
        case 6:
            printf("SpinRight\n");
            SpinRight(7000);
            break;
    }
}

```

6. Experimental Phenomena

Use Bluetooth connection within the APP to control the car. The serial port will continuously print the car's current status information.

